Structured Memory-R1: Intermediate Implementation Report

Yifan Liu **Liam Gallagher** **David Courtis** **Jiazhou Liang**

1 Problem Definition

We consider an LLM-based agent augmented with an external memory system that persists across interactions:

$$\mathcal{A} = (\pi_\theta,\ \mathcal{X},\ \mathcal{Y},\ \mathcal{M},\ \mathcal{E}),$$

- **policy** π_θ: maps an observation and a memory state to a distribution over action trajectories;
- **input (observation) space** $\mathcal{X}$: the set of all possible user queries, dialogue contexts, or environmental observations the agent may receive;
- **output (response) space** $\mathcal{Y}$: the set of all possible natural-language responses the agent may produce;
- **memory space** $\mathcal{M}$: the set of all possible external memory states the agent can read from and, optionally, write to;
- **external environment interface** $\mathcal{E}$: the set of tools, APIs, or retrieval services through which the agent can perform actions beyond pure language generation (e.g., a memory retrieval server, a search engine, or a code executor).

An *interaction* is a temporally ordered sequence of turns between a user and the agent:

$$\mathcal{I} = (t_1, t_2, \ldots, t_T),$$

where T is the total number of turns in the interaction. At turn t ($1 \le t \le T$), the following exchange occurs:

1. The user provides an input $x_t \in \mathcal{X}$ (a query, instruction, or follow-up).
2. The agent observes x_t together with the current memory state $M_t \in \mathcal{M}$.
3. The agent generates an action trajectory as a sequence of *language actions* (reasoning steps, final answers) and *environment actions* (memory queries, tool calls). The trajectory length L_t may vary per turn.
4. The agent emits a final response $y_t \in \mathcal{Y}$ extracted from τ_t.
5. The memory state may be updated: $M_{t+1} = f_{\text{update}}(M_t, \tau_t)$, where f_{update} encodes any write operations performed during the turn.

The objective is to learn a policy π_θ that controls memory access when and how memory should be queried to improve downstream task performance.

2 Structured Memory-R1

Recent work, notably Search-R1 [Jin et al., 2025] and Memory-R1 [Yan et al., 2025], has shown that RL can effectively train agents to manage flat external memory banks. A *flat memory state* is an unordered collection of N memory entries:

$$M_t^{\text{flat}} = \{m_1, m_2, \ldots, m_N\}, \quad m_i = (\text{key}_i,\ \text{attributes}_i),$$

where each entry m_i consists of a textual key, a textual attribute, and optional metadata. However, flat representations treat all memory entries as independent, discarding the hierarchical and relational structure often present in real-world user state (e.g., a dietary preference nested under a user profile, or a hotel booking linked to a specific trip). We hypothesize that exposing this structure to the policy enables more selective retrieval and reduces over-retrieval of irrelevant entries. Therefore, we proposed *Structured Memory-R1*.

Preprint. Under review.

2.1 Memory States

Building on Search-R1 and Memory-R1, our work extends reinforcement learning with verifiable rewards from external information retrieval to *structured* memory for long-context conversational AI scenarios. Specificly, a *structured memory state* at turn t is a rooted tree:

$$M_t = (V_t,\ E_t,\ r),$$

V_t is a finite set of **nodes** (memory units). Each node $v \in V_t$ carries:

- a **type label** $\ell(v) \in \mathcal{L}$ (e.g., ITINERARY, DAY, POI, SPEAKER, FACT);
- a set of **textual attributes** $\text{attr}(v) = \{(k_1, val_1), \ldots, (k_m, val_m)\}$.

$E_t \subseteq V_t \times V_t$ is a set of directed **parent–child edges** encoding hierarchical relations such as:

$$\text{ITINERARY} \rightarrow \text{VERSION} \rightarrow \text{DAY} \rightarrow \text{POI},$$

$r \in V_t$ is the designated **root node**.

2.2 Action Space

The agent's action space is $\mathcal{A} = \mathcal{A}_{\text{lang}} \cup \mathcal{A}_{\text{env}}$, where:

- $\mathcal{A}_{\text{lang}}$: **Reasoning** (`<think>`) and **Answer** (`<answer>`) generation.
- $\mathcal{A}_{\text{env}}$: **Memory Query** (`<memory>`q`</memory>`) and **Memory Update** (ADD, UPDATE, DELETE).

A memory query sends $q \in \mathcal{Q}$ to the retrieval server, which returns matching subtrees with path annotations:

$$\texttt{retrieve}(q, M_t) = \big\{(v, \text{path}(r \rightsquigarrow v), \text{subtree}(v)) \;\big|\; v \in V_t,\ \text{match}(q, v)\big\}.$$

A memory update modifies the tree via $f_{\text{update}} : \mathcal{M} \times \mathcal{T} \rightarrow \mathcal{M}$.

At turn t, the agent observes input x_t and memory state $\mathcal{M}_t$, and produces a response through a trajectory of interleaved language and memory actions:

$$\tau_t = (a_{t,1}, a_{t,2}, \ldots, a_{t,L}) \sim \pi_\theta(\cdot \mid x_t, \mathcal{M}_t).$$

2.3 Verifiable Rewards

The total reward decomposes into three verifiable components:

$$R(\tau_t) = \alpha \cdot R_{\text{syntax}}(\tau_t) + \beta \cdot R_{\text{utility}}(\tau_t) + \gamma \cdot R_{\text{efficiency}}(\tau_t),$$

$R_{\text{syntax}}(\tau_t) = \mathbf{1}[\tau_t$ uses well-formed `<memory>`, `<think>`, `<answer>` tags$]$ verifies format correctness.

$R_{\text{utility}}(\tau_t) = \lambda_{\text{EM}} \cdot \mathbf{1}[y_t = y_t^*] + \lambda_{\text{sub}} \cdot \mathbf{1}[y_t^* \subseteq y_t]$ measures answer correctness against the ground truth y_t^*. For open-ended tasks, F1 or BLEU-1 may substitute.

$R_{\text{efficiency}}(\tau_t) = -\lambda_{\text{tok}} \cdot |\text{tokens}_{\text{retrieved}}|/C - \lambda_{\text{act}} \cdot |\text{mem_actions}|$ penalizes over-retrieval, incentivizing targeted subtree access.

2.4 Policy Optimization

The *policy* π_θ is a conditional distribution over action trajectories, parameterized by the LLM weights θ:

$$\pi_\theta(\tau_t \mid x_t, M_t) = \prod_{\ell=1}^{L_t} \pi_\theta(a_{t,\ell} \mid x_t, M_t, a_{t,<\ell}),$$

where $a_{t,<\ell} = (a_{t,1}, \ldots, a_{t,\ell-1})$ denotes the actions taken so far within the current turn. The policy factorizes autoregressively at the token level within each action.

For a fixed context $(x_t, \mathcal{M}_t)$, we sample a group of K trajectories and assign each a scalar reward $r(\tau_t^{(k)})$ based on downstream criteria such as correctness, consistency, and personalization. Rather than using these rewards as absolute training signals, we optimize the policy using Group Relative Policy Optimization (GRPO), which computes a group-relative advantage for each trajectory:

$$\hat{A}^{(k)} = \frac{r(\tau_t^{(k)}) - \mu}{\sigma}, \quad \mu = \frac{1}{K}\sum_{k=1}^{K} r(\tau_t^{(k)}), \quad \sigma = \text{std}\Big(\{r(\tau_t^{(k)})\}_{k=1}^{K}\Big).$$

Given state $s = (x_t, M_t^{\text{old}})$, the manager samples K operation trajectories $\{(o^{(k)}, m'^{(k)})\}_{k=1}^{K}$, each producing an updated memory $M_t^{(k)} = f_{\text{update}}(M_t^{\text{old}}, o^{(k)}, m'^{(k)})$. The reward is whether the Answer Agent can answer correctly from $M_t^{(k)}$: trajectories that produce well-consolidated memory receive positive advantage, while those that fragment or overwrite useful information receive negative advantage.

3 Progress to Date

3.1 System Architecture

Our implementation faithfully follows the Memory-R1 two-agent architecture [Yan et al., 2025], with extensions for tree-structured memory. The system consists of five main components:

Memory Data Model. We implement two memory representations:

1. **Flat Memory** (Memory-R1): A list-based memory bank with keyword and embedding-based top-k retrieval, where each entry is a text string with ID and optional metadata. Supports ADD, UPDATE, DELETE, and NOOP operations.
2. **Structured Memory** (Struct Memory-R1): A rooted tree where each node has a type, textual attributes, and parent–child edges. ADD operations specify a `parent_id` to position new nodes in the hierarchy. This supports compositional hierarchies (e.g., `Itinerary` → `Day` → `POI`) and session-based hierarchies (e.g., `Dialogue` → `Session` → `MemoryEntry`).

Memory Manager Agent (Stage 1). Following the Memory-R1 paper, the Memory Manager is an RL-trained agent that processes each dialogue turn and decides how to update the memory bank. For each extracted fact and its related retrieved memories, the manager outputs one of four operations:

- ADD: Insert a new entry (flat) or a new node at a specified parent (tree).
- UPDATE: Merge new information into an existing entry, preserving the more detailed version.
- DELETE: Remove an outdated or contradictory entry.
- NOOP: No change needed (fact already present or irrelevant).

The output is structured JSON, enabling deterministic parsing and application to the memory bank. For the tree variant, ADD operations include `parent_id` and `node_type`, requiring the manager to make structural placement decisions.

Answer Agent (Stage 2). The Answer Agent receives a question and top-60 retrieved memories from the bank, applies a *Memory Distillation* policy to select the most relevant entries, and generates a concise answer (<5–6 words). This two-step filtering mitigates the "lost in the middle" problem [Liu et al., 2024] where LLMs degrade when relevant information is buried among distractors.

Two-Stage Training. Both agents are trained independently with GRPO:

1. **Memory Manager training**: The Answer Agent is frozen. The reward for each candidate operation set is the EM score of the frozen Answer Agent's response given the updated memory bank.
2. **Answer Agent training**: The Memory Manager is frozen. Memory banks are rebuilt using the trained manager, and the Answer Agent is trained with EM reward between its predicted and gold answers.

This decoupled training avoids attribution ambiguity while enabling both agents to co-adapt across alternating training phases.

Data Construction Pipeline. Following the paper's Algorithms 1–2, we use GPT-4o-mini to extract facts from each LoCoMo dialogue turn, build temporal memory banks from preceding turns, and create separate training parquet files for both agents. Retrieval uses embedding similarity over the memory bank.

3.2 Data Curation

We prepare evaluation data from two sources:

Semantic XPath Domains. Following the evaluation setup in Semantic XPath [Liu et al., 2026], we curate structured memory trees for three conversational AI domains:

- **Travel Itinerary**: A 7-day San Diego trip plan with the schema `Itinerary` → `Version` → `Day` → `POI`. Each POI has attributes including name, time, category, description, and cost.
- **To-Do List**: A structured task list with 5 categories (Work, Household, Personal, Finance, Social), each containing projects with tasks annotated with status, deadline, and priority.
- **Meal Kit Recommendation**: A 7-day family meal plan with per-member dietary restrictions and 3 options per meal time, annotated with ingredients, calories, and allergen information.

Each domain includes 20 curated QA pairs spanning single-hop, multi-hop, and reasoning question types. These data files serve both as the structured memory state for the memory server and as the source of training/evaluation QA pairs.

Structured LoCoMo. We convert the LoCoMo benchmark [Maharana et al., 2024], a long multi-session dialogue dataset, into structured memory trees. The dataset consists of 10 long-term conversations, each annotated with question-answer pairs and containing: sessions of speaker turns with timestamps, per-session event summaries, per-session prose summaries, and per-session observations : speaker-attributed, turn-grounded atomic claims such as *"Caroline is researching adoption agencies"* (D2:8). We use the following schema:

```
Conversation
├── Speaker (name)
│   └── Observation (text, source_dia_id, session_id)
└── Session (session_id, datetime)
    ├── Summary (text)
    ├── Event (speaker, description)
    └── Turn (dia_id, speaker, text, image_caption)
```

`Speaker` nodes aggregate persistent observations about each participant across all sessions. `Session` nodes preserve the raw conversational record alongside two levels of derived structure: dense prose `Summary` nodes, which enable efficient session-level semantic filtering, and sparse `Event` nodes capturing key milestones. The dataset's gold observations map directly to `Observation` nodes, providing supervision for memory construction without requiring LLM-based extraction.

This schema supports two natural query patterns. *Profile queries* retrieve persistent facts about a speaker directly from their observation nodes:

LoCoMo Profile Query. The query for *"How long has Melanie been practicing art?"* can be translated into:

```
//Speaker[@name ≈ "Melanie"]//Observation[node ≈ "art practice duration"]
```

This scores all of Melanie's observations against "art practice duration", retrieving *"Melanie has been into art for seven years, finding a passion for painting and pottery"* (D16:8).

Temporal queries use `Summary` nodes as a predicate to efficiently narrow to the relevant session before retrieving specific turns within it:

LoCoMo Temporal Query. The query for *"When did Caroline pass her adoption interview?"* can be translated into:

```
//Session[Summary ≈ "adoption interview"]//Turn[@speaker="Caroline" AND node ≈
                                "passed interview"]
```

This scores each `Session` node by the semantic relevance of its `Summary` child against "adoption interview", selecting Session 19, then descends to retrieve Caroline's turn confirming she passed the adoption agency interviews, correctly grounding the answer to October 2023.

3.3 Non-RL Baselines

In addition to the RL-trained memory agents, we have implemented two non-RL baselines for comparison:

In-Context Memory. The entire memory state is serialized as text and appended to the LLM's input prompt. This provides an upper bound on information availability but scales poorly with memory size: as shown by

Semantic XPath results [Liu et al., 2026], token usage grows linearly with turns while pass rate degrades due to the "lost in the middle" phenomenon [Liu et al., 2024].

Semantic XPath. The LLM generates an XPath-style query over the structured memory tree, which is then executed by the Semantic XPath engine to retrieve only the relevant substructure. This provides a strong non-RL baseline for structured memory access, achieving comparable pass rates to in-context memory while using only 9.1% of the tokens [Liu et al., 2026]. We use the results reported in the Semantic XPath paper as reference baselines.

3.4 Signs of Life: Preliminary Trajectory Analysis

To demonstrate that our framework shows signs of life, we compare single trajectories from the zero-shot baseline (Qwen2.5-3B without RL) and our memory-augmented system on representative queries from the itinerary domain.

Example: Zero-Shot Baseline.

> **Q:** Which day is packed with conference sessions?
> **Agent (no memory):** `<think>` I need to think about which day has conference sessions. Based on typical conference schedules, it would likely be a middle day. `</think>` `<answer>` Day 3 `</answer>`
> **Result:** Incorrect (ground truth: Day 2). The model hallucinates without access to the actual itinerary.

Example: Memory-Augmented Agent.

> **Q:** Which day is packed with conference sessions?
> **Agent:** `<think>` I need to find which day has the most conference-related activities. Let me query the memory for conference sessions. `</think>` `<memory>` conference sessions day `</memory>`
> **Memory returns:** `<information>` Memory 1 (Path: /Itinerary/Version/Day): [Day] day_number: 2 | date: 2026-08-11 | theme: Conference Day 1 ... ACL Welcome Reception ... Paper Presentations ... Poster Session `</information>`
> **Agent:** `<think>` Day 2 has multiple conference events: welcome reception, paper presentations, poster session, and networking dinner. This is the most conference-packed day. `</think>` `<answer>` Day 2 `</answer>`
> **Result:** Correct. The agent queries memory, retrieves the relevant day, and reasons over the results.

Preliminary Results. Table 1 summarizes preliminary single-run results on a subset of our curated itinerary QA pairs. Even before RL training, the memory-augmented agent (using the structured memory server) outperforms the zero-shot baseline by a substantial margin, demonstrating that the memory retrieval interface is functional and provides useful information to the LLM. The in-context baseline achieves the highest accuracy but at 5× the token cost. After a small number of GRPO training steps, we expect the RL-trained agent to further improve its memory query strategies.

Table 1: Preliminary results on itinerary domain QA (20 questions, single run). The memory-augmented agent accesses structured memory via the retrieval server. Token usage is measured per query.

Method	Sub-EM	Pass Rate	Avg. Tokens
Zero-Shot (no memory)	0.15	15.0%	512
In-Context Memory	0.70	70.0%	12,136
Flat RAG	0.30	30.0%	3,250
Semantic XPath (Entail)	0.85	85.0%	5,161
Memory-R1 Flat (ours, no RL)	0.35	35.0%	3,500
Struct Memory-R1 (ours, no RL)	0.55	55.0%	4,200

These results are based on the non-RL versions of our agents (zero-shot memory access), with reference baselines drawn from Semantic XPath [Liu et al., 2026] results. As we proceed with GRPO training, we expect the RL-trained agents to improve memory query quality and approach the Semantic XPath baseline performance while maintaining the adaptability that RL provides.

4 The Plan Moving Forward

4.1 RL Training

Our immediate next step is to run full GRPO training for both the flat memory and structured memory agents. Following Search-R1's training recipe, we will:

1. Train with Qwen2.5-3B as the base model for 500 GRPO steps with $K = 5$ trajectory samples per query.
2. Evaluate on the Semantic XPath domain QA pairs and the structured LoCoMo benchmark.
3. Scale to Qwen2.5-7B to study the effect of model capacity on memory management quality.

4.2 Evaluation Plan

For the final report, we plan to present results averaged across multiple runs on:

- **Semantic XPath Domains**: Pass rate and token usage on all three domains (itinerary, to-do, meal kit), comparing in-context memory, flat RAG, Semantic XPath, flat Memory-R1, and Struct Memory-R1.
- **Structured LoCoMo**: F1, BLEU-1, and Sub-EM on the LoCoMo QA benchmark, following the evaluation protocol from Memory-R1 [Yan et al., 2025].
- **Ablations**: Effect of structured vs. flat memory, number of training steps, and GRPO group size K.

4.3 Planned Improvements

Based on the preliminary analysis and newly completed two-agent implementation, we identify several directions for the final report:

1. **Dense Embedding Retrieval**: Upgrade the keyword-based retrieval to dense embedding search using sentence-transformers for both flat and structured memory.
2. **Semantic XPath Query Generation**: Train the RL agent to output structured XPath-style queries instead of free-form natural language, enabling more precise tree navigation.
3. **Full Two-Agent Evaluation**: With the Memory Manager (ADD/UPDATE/DELETE/NOOP) and Answer Agent (Memory Distillation) now implemented, run full GRPO training on LoCoMo and evaluate across all three benchmarks (LoCoMo, MSC, LongMemEval) following the Memory-R1 protocol.
4. **Tree vs. Flat Manager Ablation**: Compare how the tree-aware Memory Manager (which must decide *where* to insert nodes) performs against the flat manager on structural QA tasks requiring multi-hop tree traversal.

4.4 Timeline

- **Weeks 1–2**: Complete GRPO training for both memory types; generate full evaluation results.
- **Weeks 3–4**: Run ablation studies; implement dense retrieval upgrade.
- **Weeks 5–6**: Write final paper; prepare presentation materials.

References

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

Yifan Simon Liu, Ruifan Wu, Liam Gallagher, Jiazhou Liang, Armin Toroghi, and Scott Sanner. Semantic XPath: Structured agentic memory access for conversational AI. *Under Review*, 2026.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024.

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Kristian Kersting, Jeff Z Pan, Hinrich Schütze, et al. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.